

Splitwaisito

Ficha técnica del producto

Materia: Aplicaciones Móviles · Trabajo práctico final · Equipo de 6 integrantes

1. Concepto y propósito

Splitwaisito es un clon gratuito de Splitwise para dividir y llevar el control de gastos compartidos entre grupos (viajes, convivencia, eventos): carga de gastos, categorización, y división o registro de deudas según convenga. Incluye funcionalidades que en apps de referencia del mercado son pagas, sin costo.

Objetivo del proyecto: cumplir con el trabajo práctico final de la materia y, a la vez, construir una app real que el equipo pueda usar entre sí y sumar como pieza de portafolio — no un entregable puramente académico.

Diferenciador principal: un botón para saldar deudas sin tener que buscar a mano el dato de contacto del acreedor. El monto sale directo del balance de la deuda (Firestore); el e-mail del acreedor, de su perfil.

2. Relación con la consigna

La idea central — clonar Splitwise y liberar sus funciones premium — se ajusta a lo que pide la consigna, con dos precisiones:

- **Tecnología:** la consigna original mencionaba Expo + React Native + Firebase, sin PWA. El equipo consultó al docente y confirmó que una PWA es válida. Se optó por PWA porque elimina la fricción de instalación vía store y permite que cualquier integrante o usuario final la use desde el navegador, en cualquier dispositivo — ver sección 5.
- **Módulo de e-commerce:** se cubre con el flujo de pago / saldado de deudas (pantalla «Saldar deuda»), que la consigna acepta como una de las variantes válidas del requisito (pagos simulados, reservas o compras).

3. Alcance del MVP

3.1 Pantallas núcleo (6, mínimo pedido por la consigna)

- **Login / Registro** — Firebase Auth, con e-mail o Google.
- **Tus grupos** — lista de grupos del usuario con balance neto de cada uno.
- **Detalle de grupo** — pestañas Gastos / Deudas simplificadas.
- **Cargar gasto** — monto, categoría, quién pagó, división, foto del ticket (cámara) y ubicación (GPS).
- **Saldar deuda** — abre «Enviar dinero» de Mercado Pago con el monto y el e-mail del acreedor ya copiados.
- **Perfil** — datos para cobrar (alias / e-mail de Mercado Pago) y moneda preferida.

3.2 Cámara y GPS

Cámara: foto del ticket como respaldo del gasto (sin OCR en esta primera versión — queda en el backlog de extras). Ubicación: se pre-detecta automáticamente al cargar un gasto y se muestra

como mini-mapa con una fila de lugares cercanos sugeridos, para que el usuario ajuste si el detectado no es exacto.

3.3 Monedas

Tres monedas fijas en esta primera versión: pesos argentinos (ARS), dólares estadounidenses (USD) y reales brasileños (BRL) — cubren los casos de uso reales del equipo (gastos compartidos dentro y fuera del país) sin la complejidad de un sistema de monedas genérico con conversión en vivo.

4. Categorías de gasto — justificación

El formulario de carga de gasto usa cuatro categorías: Alojamiento, Comida, Transporte y Otro. Se eligió este conjunto reducido, en vez de una lista larga y granular, por tres razones:

- Cubre la gran mayoría de los gastos reales de un grupo (viaje, convivencia, evento) sin obligar a elegir entre opciones ambiguas o redundantes.
- Mantiene el formulario de carga rápido de completar — el foco del producto es que cargar un gasto tome segundos, no que se navegue un árbol de categorías.
- «Otro» funciona como categoría de escape: ningún gasto queda bloqueado por no encajar en una categoría predefinida, y sirve como señal de qué categorías nuevas agregar más adelante si se repiten.

Categorías adicionales (por ejemplo, entretenimiento o compras) quedan como posible ampliación futura, condicionadas a que el uso real del equipo muestre que «Otro» se usa demasiado seguido.

5. Stack técnico

Elegido a partir del stack ya probado por Lucas en un proyecto personal previo (Flujo — React + Vite + TypeScript), adaptado a los requisitos propios de Splitwaisito: base de datos Firebase (pedida por la consigna) en vez de un backend propio, y capa de PWA para instalación sin fricción.

Capa	Tecnología	Motivo
Frontend	React 18 + Vite + TypeScript (strict)	Stack ya probado por el equipo en un proyecto previo (Flujo). Vite da un dev server y build rápidos.
Estilos	Tailwind CSS	Mismo sistema de diseño que Flujo; permite iterar rápido sobre la paleta definida en los wireframes.
Estado	Zustand + persist	Estado global simple, con cache local para que la app responda instantáneo mientras sincroniza con Firestore.
Navegación	React Router v6	Estándar de facto para SPAs en React; sin curva de aprendizaje adicional para el equipo.
Fechas	date-fns	Manejo de fechas para el calendario de gastos y vencimientos.
Íconos	lucide-react	Set de íconos consistente, liviano, usado en los wireframes.

Gráficos	recharts	Para el extra «gráfico de gastos por categoría» del backlog.
PWA	vite-plugin-pwa (Workbox)	Genera manifest.json y service worker automáticamente. Instalable en cualquier dispositivo desde el navegador, sin store — objetivo central del proyecto: cero fricción de acceso.
Autenticación	Firebase Auth (e-mail + Google)	Requisito de la consigna. Proveedor de Google porque ya está contemplado en el wireframe de login.
Base de datos	Firestore	Requisito de la consigna. Sin servidor propio que mantener — clave para un equipo de 6 sin backend dedicado.
Archivos	Firebase Storage	Guarda las fotos de los tickets de gasto.
Lógica de servidor	Cloud Functions	Corre la función que simplifica las deudas de un grupo (quién le debe a quién con el mínimo de transferencias).
Cámara	input file (capture) / getUserMedia	API estándar del navegador — sin permisos nativos ni dependencias adicionales.
Ubicación	navigator.geolocation	API estándar del navegador, con geocodificación inversa para mostrar el nombre del lugar.

6. Arquitectura y modelo de datos

Arquitectura cliente-servidor liviana: el frontend (PWA) habla directo con Firebase — no hay backend propio que mantener. Colecciones principales en Firestore:

- **usuarios** — perfil, alias / e-mail de Mercado Pago, moneda preferida.
- **grupos** — nombre, integrantes, tipo (viaje, convivencia, evento).
- **grupos/gastos** — subcolección por grupo: monto, categoría, quién pagó, división, foto del ticket, ubicación.

Reglas de seguridad de Firestore: solo los integrantes de un grupo pueden leer o escribir sus gastos; cada usuario edita únicamente su propio perfil. Una Cloud Function calcula, por grupo, la forma de saldar todas las deudas con la menor cantidad de transferencias posible (el mayor deudor le paga al mayor acreedor, y así sucesivamente), en vez de que todos le deban a todos.

7. Mecanismo de pago (saldar deudas)

Se investigó en profundidad si Mercado Pago permite abrir un flujo de transferencia con el destinatario y el monto precargados (spike documentado aparte, con pruebas en dispositivo real, decodificación de QR e inspección del código del sitio de Mercado Pago). Conclusión: no existe una API pública ni un deep link documentado para precargar una transferencia P2P; tampoco hay ningún endpoint de Mercado Pago Developers para enviar dinero — solo para cobrar.

El mecanismo que quedó en el MVP: el botón «Saldar deuda» copia automáticamente al portapapeles el e-mail del acreedor, muestra el monto, y abre el flujo web «Enviar dinero» de Mercado Pago (verificado de punta a punta con una transferencia real). El usuario pega el e-mail, confirma el monto, y listo. Splitwaisito nunca mueve dinero directamente — solo evita que el usuario tenga que buscar el dato de contacto a mano.

8. Diseño visual

Paleta clara: fondo blanco / gris muy claro, acento principal verde (no saturado) y acento secundario celeste — sin modo oscuro y sin imitar la identidad visual de la competencia. La interfaz no menciona a Splitwise en ningún texto de cara al usuario, para que el producto se sostenga con identidad propia.

9. Gestión del proyecto y flujo de trabajo

Equipo de 6 personas, sin organización por sprints (la disponibilidad de cada integrante varía mucho por día y por semana). Tablero Kanban en Trello con 5 estados (Backlog, Asignada, En progreso, En revisión, Hecho) y 25 tarjetas del MVP, autoasignables, cada una con historia de usuario o instrucción técnica, tamaño estimado y dependencias.

Flujo de trabajo en Git: cada tarea se resuelve en una rama propia desde develop, se sube por Pull Request, la revisa un compañero, y se mergea a develop. El pase de develop a main se hace solo en momentos de entrega coordinados, también por Pull Request. Un workflow de GitHub Actions (build de producción con Vite) corre en cada Pull Request, y la rama main tiene protección para que nadie pueda pushear directo — todo pasa por revisión.

10. Proceso de creación y uso de inteligencia artificial

Gran parte del proceso de definición de producto, investigación técnica, diseño de wireframes, planificación de tareas y documentación de este proyecto se hizo en colaboración con un asistente de IA (Claude). Cada uso quedó registrado con la herramienta utilizada, la finalidad concreta y los prompts clave, en la planilla «Planilla de uso de IA» que acompaña esta ficha técnica — tal como pide la consigna.

A modo de resumen, la IA se usó como asistente de investigación y producción para: la validación técnica del mecanismo de pago con Mercado Pago (con evidencia verificable: pruebas en dispositivo real, decodificación de código, e inspección de la documentación oficial), la generación de los wireframes de las 6 pantallas del MVP, la redacción del backlog de 25 tareas y la guía de flujo de trabajo para el equipo, la configuración de CI/CD, y la redacción de esta misma ficha técnica. En todos los casos, las decisiones de producto (qué construir, qué prescindir, qué priorizar) las tomó el equipo — la IA investigó, propuso opciones y documentó, pero no decidió por su cuenta.

11. Roadmap

Fuera del alcance del MVP, quedan documentadas como extras de valor agregado (no bloquean la entrega): OCR de tickets, notificaciones push de recordatorio de deuda, invitar integrantes por contactos del teléfono, gráfico de gastos por categoría, exportar gastos a PDF/Excel, y recurrencia automática de gastos fijos.